?

DISTRIBUTION OF ELECTRIC POWER FOR AGRICULTURE

Ministry of Power Portal — Rajasthan

Project Domain:	Public Utility & E-Governance
Backend Framework:	Laravel 10.10 (PHP 8.1+)
Frontend Stack:	Blade Templates & Tailwind CSS
Database Engine:	MySQL 8.0
Document Type:	Comprehensive Viva Preparation, MCQ Bank & Architecture Guide

Developed By: Ravi Ranjan

GitHub: github.com/Raviranjan010

Purpose: Academic Thesis & Project Defense

1. PROJECT OVERVIEW & ARCHITECTURE

The **Agricultural Electric Power Distribution Portal** is a specialized e-governance platform designed for the Rajasthan Ministry of Power. Its primary goal is to digitize and manage the electrical supply infrastructure dedicated to the agricultural sector. The system coordinates interaction between four core user groups: **Administrators**, **Sub-Divisional Officers (SDOs)**, **Field Linemen**, and **Farmers**.

Key Modules:

- **Connection Lifecycle Management:** Farmers request connections (e.g., tubewell, drip irrigation); SDOs review, assign tariffs, and activate meters.
- **Metering Pipeline:** Linemen submit physical readings with mandatory GPS coordinates and photo uploads to ensure authenticity. SDOs verify these readings.
- **Automated Billing Engine:** Auto-calculates Energy Charges, Fixed Charges, and Taxes, while subtracting applicable government subsidies.
- **Crowd-Sourced Outage Tracking:** Integrates community outage reporting that automatically flags high-priority grid issues when 3+ reports occur in the same zone within 30 minutes.
- **Subsidy Administration:** Processes farmer claims for schemes like PM-KUSUM, validating uploaded documents.

MVC Design Pattern Implementation:

This project uses the Model-View-Controller pattern natively supported by Laravel:

- **Models (`app/Models/`):** Define the schema structure, casts, and database relationships (e.g., `User.php`, `Connection.php`, `Bill.php`).
- **Views (`resources/views/`):** Blade templates rendering responsive layouts. Folders are structured by role: `admin/`, `officer/` (SDO), `lineman/`, `farmer/`, and `auth/`.
- **Controllers (`app/Http/Controllers/`):** Handle processing. Key controllers include `AuthController.php`, `FarmerController.php`, `OfficerController.php`, `LinemanController.php`, and `AdminController.php`.

2. CORE WORKFLOWS (DEEP-DIVE)

A. How the Login Process Works

> The login flow secures role-based entry points.

1. **Request Entry:** The user accesses `/login` (route named `login` in `routes/web.php`), calling `showLogin()` in `AuthController.php`, which returns the view `auth.login`.
2. **Form Submission:** The login POST request is routed to `login()` in `AuthController.php`, wrapped with the `throttle:5,1` middleware to block brute force logins (limits users to 5 attempts per minute).
3. **Validation:** The inputs are validated to ensure a valid email format and presence of a password.
4. **Database Authentication:** Laravel calls `Auth::attempt(['email', 'password'])`. The query driver checks the `users` table for the email, gets the bcrypt hashed password, and performs a secure hash match.
5. **Active Check:** If the hash matches, the system retrieves the User object and checks if `$user->is_active` is true. If false, the session is cleared (`Auth::logout()`) and the user is redirected back with an error.
6. **Session Protection & Redirect:** If active, Laravel calls `$request->session()->regenerate()` to generate a new session ID (combating session fixation). A `match` statement evaluates `$user->role` and redirects to the corresponding dashboard:
   - `admin` ? `admin.dashboard` (/admin/dashboard)
   - `sdo` ? `officer.dashboard` (/officer/dashboard)
   - `lineman` ? `lineman.dashboard` (/lineman/dashboard)
   - `farmer` ? `farmer.dashboard` (/farmer/dashboard)

B. How the Notification System Works

> The notification engine keeps farmers and SDOs informed using database persistence and AJAX polling.

1. **Creation:** When a transaction completes (e.g., SDO approves a connection), the code invokes the notify method:

```
$user->notify(new \App\Notifications\RealTimeNotification($title, $message, $url, $icon));
```

2. **Storage:** The `via()` method of `RealTimeNotification.php` returns `['database']`. Laravel serializes the notification's array values (title, message, redirect URL, icon) into JSON and stores them in the `notifications` table:

```
id (UUID) | type | notifiable_type | notifiable_id | data (JSON) | read_at | timestamps
```

3. **Frontend AJAX Engine:** In the layout file `app.blade.php`, a JavaScript block runs a `fetchNotifications()` method on DOM load. It sets up polling:

```
setInterval(fetchNotifications, 8000); // Polls every 8 seconds
```

4. **Backend Retrieval:** The JavaScript polls `/notifications/poll`, which maps to `poll()` in `NotificationController.php`. The controller fetches the user's unread notifications:

```
$unreadCount = $user->unreadNotifications()->count();
$notifications = $user->unreadNotifications()->limit(5)->get();
```

   It returns these as a JSON response containing the message details and time format (e.g., "5 minutes ago").

5. **Toast Rendering:** The JavaScript reads the JSON. It matches notification IDs against a set stored in browser `localStorage`. If a new ID is found, it adds it to the known set and triggers an animated, slide-in toast card at the bottom-right corner of the screen. Clicking the card marks it as read and redirects the user.

2. CORE WORKFLOWS (CONTINUED)

C. How Language Changing Works (Localization)

> The current portal codebase contains UI comments/hooks for a Language Selector in the layout, but remains statically in English. To implement a dynamic multi-language (English / Hindi) toggle, we use Laravel's localization architecture.

Here is the standard, best-practice way to implement dynamic language changing in Laravel:

Step 1: Create Translation Files

In Laravel 10, translations are stored in the root `lang/` directory. We create JSON files for translation matching:

- `lang/en.json` **(English):**

```
{ "dashboard": "Dashboard", "welcome": "Welcome, :name", "bills": "Bills & Payments" }
```

- `lang/hi.json` **(Hindi):**

```
{ "dashboard": "????????", "welcome": "?????? ??, :name", "bills": "??? ?? ??????" }
```

Step 2: Create a Language Controller and Switcher Route

We register a route in `routes/web.php` to capture the user's language selection:

```
Route::get('/lang/{locale}', [LanguageController::class, 'switchLang'])->name('lang.switch');
```

In the controller, we save the chosen locale (e.g., 'en' or 'hi') in the user's session:

```
public function switchLang($locale) {
    if (in_array($locale, ['en', 'hi'])) {
        session(['app_locale' => $locale]);
    }
    return back();
}
```

Step 3: Register SetLocale Middleware

To apply the language setting for every request, we write a custom middleware `SetLocaleMiddleware.php`:

```
public function handle(Request $request, Closure $next) {
    $locale = session('app_locale', config('app.locale'));
    App::setLocale($locale);
    return $next($request);
}
```

This middleware is registered in `app/Http/Kernel.php` under the `web` middleware group.

Step 4: Update Blade Views

We replace static strings in Blade views with the translation helper `__('Key')` or blade directives:

```
<a href="...">{{ __('dashboard') }}</a>
<h2>@lang('welcome', ['name' => Auth::user()->name])</h2>
```

3. CONCEPT COMPARISONS (DESIGN RATIONALE)

Q1. Why did we use Eloquent ORM instead of Raw SQL Queries or Query Builder?

> **Eloquent ORM** is Laravel's ActiveRecord implementation. We selected Eloquent over raw SQL because:
>
> 1. **Readability and Syntactic Simplicity:** Writing `$user->connections` is cleaner than writing `SELECT * FROM connections WHERE consumer_id = ?`.
> 2. **Relationship Management:** Eloquent makes relationship definitions (`hasMany`, `belongsTo`) easy, allowing us to eager-load records and avoid N+1 query problems via `$connection->load('consumer')`.
> 3. **SQL Injection Protection:** Eloquent automatically utilizes PDO parameter bindings for all queries, protecting the database against injection attacks.
> 4. **Events and Mutators:** Eloquent model lifecycle hooks (like `creating`, `updating`) allow us to track audit records easily.
>
> *Trade-off:* Raw SQL is slightly faster for complex reports, but for standard operations, Eloquent's benefits outweigh the overhead.

Q2. Why did we use Session-Based Authentication instead of JWT (JSON Web Tokens)?

> This application is a server-rendered web portal. We chose **Session-Based Authentication** because:
>
> 1. **Implicit Security:** Sessions store user state on the server, while the client only holds an encrypted cookie. JWTs must be stored on the client side (localStorage), making them vulnerable to Cross-Site Scripting (XSS) attacks.
> 2. **Immediate Revocation:** In a session-based system, if an Admin deactivates a user, we can call `Auth::logout()` or invalidate their session on the server. With standard JWTs, tokens remain valid until they expire, unless a complex blacklist is implemented.
> 3. **Laravel Native Support:** Laravel provides session authentication, secure CSRF matching, and session regeneration utilities out of the box, reducing custom code risks.
>
> *Trade-off:* JWTs are better for stateless REST APIs, but session authentication is ideal for a server-rendered Blade template application.

Q3. Why did we use Database Transactions with row-level locks (lockForUpdate) instead of basic query executions?

> In workflows like generating meter numbers (`MT-XXXXX`) or connection suffixes (`KV-CN-XXXXX`), multiple linemen or farmers could submit requests concurrently.
>
> - If we query the database using a simple `max()` query and save the record in two separate requests, a race condition can cause both requests to select the same number, resulting in duplicate entries.
> - By wrapping the logic inside `DB::transaction(function() { ... })` and calling `lockForUpdate()`, the database places an exclusive read/write lock on the target rows. Any concurrent requests must wait until the current transaction commits or rolls back. This ensures strict sequence numbering.

4. DIRECTORY MAPPING & CUSTOMIZATION GUIDE

A. Where is What? (Core File Locations)

Feature Module	Controller & Path	Model & Views
Authentication / Login	`app/Http/Controllers/AuthController.php` Handles login, logout, and registration.	Model: `app/Models/User.php` Views: `resources/views/auth/`
SDO Billing / Approvals	`app/Http/Controllers/OfficerController.php` Contains connection approvals and bill calculations.	Models: `Bill`, `Connection` Views: `resources/views/officer/`
Farmer Portal / Outages	`app/Http/Controllers/FarmerController.php` Handles connection requests and crowd-sourced outages.	Models: `OutageReport`, `Complaint` Views: `resources/views/farmer/`
Lineman Operations	`app/Http/Controllers/LinemanController.php` Handles meter readings and updating complaints.	Model: `app/Models/MeterReading.php` Views: `resources/views/lineman/`
Notification Engine	`app/Http/Controllers/NotificationController.php` Handles unread count AJAX polling.	Class: `app/Notifications/RealTimeNotification.php`

B. How to Make Changes (Code Customizations)

Scenario 1: How do you change the billing Tax Rate from 5% to 8%?

> Open `app/Http/Controllers/OfficerController.php`. Under the `generateBills` method, locate the line calculating tax:
>
> ```
> // OLD CODE:
> $tax = ($ec + $fc) * 0.05;
>
> // NEW CODE:
> $tax = ($ec + $fc) * 0.08;
> ```
>
> **Result:** Any future bills generated by SDOs will apply an 8% tax rate, updating the stored net payable amount in the `bills` table.

Scenario 2: How do you change the Outage Crowd-Source alert limit from 3 to 5 reports?

> Open `app/Http/Controllers/FarmerController.php`. Navigate to the `reportOutage` method and look for the check:
>
> ```
> // OLD CODE:
> if ($reportCount >= 3) { ... }
>
> // NEW CODE:
> if ($reportCount >= 5) { ... }
> ```
>
> **Result:** The system will now require at least 5 unique farmers in the same zone to file outage reports within 30 minutes before generating an automated high-priority ticket and SDO notification.

Scenario 3: How do you enforce a minimum password length of 8 instead of 6?

> Open `app/Http/Controllers/AuthController.php`. In both `register` and `resetPassword` methods, modify the validation array:
>
> ```
> 'password' => 'required|min:8|confirmed'
> ```
>
> **Result:** Registering farmers or resetting passwords will fail if the input password is less than 8 characters, displaying a validation error.

5. TECHNICAL VIVA QUESTIONS & ANSWERS

Q1. What version of Laravel does this project use? What are its key requirements?

> The project runs on **Laravel 10** and requires **PHP 8.1** or higher. The framework leverages PHP 8 features like constructor promotion, union types, and the `match` expression.

Q2. What is the role of Middleware in Laravel, and how is it used in this project?

> Middleware acts as a request filter. In our project, we use:
>
> - `auth`: Ensures only logged-in users access internal portals.
> - `role:admin|sdo|farmer|lineman`: Custom middleware restricting routes to specific user types. If a user tries to access a route assigned to another role, the middleware redirects them to their correct home dashboard.
> - `throttle:5,1`: Limits API endpoints (login/register/password-reset) to 5 attempts per minute to block automated dictionary attacks.

Q3. How is the database seeded with dummy records? What command do we run?

> We use seeders and model factories in `database/seeders/DatabaseSeeder.php` to populate default tariffs, zones, SDOs, linemen, and farmers. We run:
>
> ```
> php artisan db:seed
> ```
>
> This populates our schema with demo credentials immediately after running migrations.

Q4. How does Laravel protect against Cross-Site Request Forgery (CSRF) in this project?

> Laravel's `VerifyCsrfToken` middleware checks all incoming POST, PUT, and DELETE requests. In our blade forms, we include the `@csrf` directive. This renders a hidden input field containing an encrypted token. When submitted, the middleware validates this token against the one stored in the user's session.

Q5. Explain the billing formula used by the SDO to generate bills.

> The formula calculated inside `generateBills()` is:
>
> ```
> Energy Charges (EC) = Units Consumed × Tariff Rate Per Unit
> Fixed Charges (FC) = Sanctioned Load (kW) × Fixed Tariff Charge Per kW
> Taxes = (EC + FC) × 5%
> Net Payable = Max( 0, (EC + FC + Taxes) - Subsidy Amount )
> ```

Q6. What happens if a farmer has no approved subsidy? How does the billing engine behave?

> If a farmer has no approved subsidy request but has an active agricultural connection, the billing engine automatically searches for active government schemes (like KUSUM or solar subsidies). It applies the best matching scheme to calculate the bill and logs this under the `consumer_subsidies` table for transparency.

Q7. How do you handle file uploads, such as subsidy verification documents or meter photos?

> We use Laravel's `Storage` facade:
>
> ```
> $path = $request->file('avatar')->store('avatars', 'public');
> ```
>
> This saves the file inside `storage/app/public/avatars/` and returns the relative path. We run the command `php artisan storage:link` to create a symlink from `public/storage` to `storage/app/public`, allowing the files to be accessed via the browser.

5. TECHNICAL VIVA QUESTIONS & ANSWERS (CONTINUED)

Q8. How does the crowd-sourced outage tracking system prevent spam?

> In `FarmerController.php`, we enforce a time restriction of 30 minutes. If a farmer attempts to submit an outage report, the system queries the `outage_reports` table for a report from that user within the last 30 minutes:
>
> ```
> $recent = OutageReport::where('farmer_id', $user->id)->where('reported_at', '>=', now()->subMinutes(30))->exists();
> ```
>
> If a record exists, the submission is rejected with a validation error.

Q9. How are the usage charts on the farmer and admin dashboards rendered?

> We use **Chart.js** on the client side. The frontend makes an AJAX fetch request to `/farmer/usage/chart-data`, which returns a JSON array containing labels (the last 12 months) and values (total units consumed by the farmer's active connections). Chart.js parses this JSON to render the line/bar charts.

Q10. What is the role of DB migration files in Laravel? What is the command to roll them back?

> Migration files act as version control for the database schema, defining database tables, columns, and indexes. To rollback the last migration batch, we run:
>
> ```
> php artisan migrate:rollback
> ```
>
> To clear the database and re-run all migrations from scratch, we use:
>
> ```
> php artisan migrate:fresh
> ```

Q11. Why do we store the password in the database as a hash? What hashing algorithm does Laravel use?

> Storing passwords in plain text is a severe security risk. Hashing is a one-way function, meaning a hashed password cannot be decrypted. Laravel uses the **bcrypt** hashing algorithm by default (configured via the `password` cast set to `hashed` in the User model).

Q12. What is the difference between eager loading and lazy loading in Eloquent?

> - **Lazy Loading:** Laravel only queries related data when the relationship property is accessed. This can trigger the N+1 query issue (1 query for connection list + N queries for each connection's farmer details).
> - **Eager Loading:** We pre-fetch related data using the `with()` method (e.g., `Connection::with('consumer')->get()`). This reduces queries to just 2: one for the connections and one for the users.

Q13. How does the payment simulation work when Razorpay keys are not set?

> In `FarmerController.php`, the system checks if the Razorpay API keys are configured in the `.env` file. If they are absent, the system skips the Razorpay API call, returns a fallback payment view, and generates a local transaction token:
>
> ```
> $txnId = 'TXN-' . now()->format('YmdHis') . '-' . $bill->id;
> ```
>
> This marks the bill as paid in the database without calling Razorpay, facilitating local development.

Q14. How are Admin audit logs stored, and what columns do they contain?

> Whenever an administrator activates/deactivates a user, adds a tariff, or changes a zone, a record is added to the `audit_logs` table. The model stores:
>
> - `user_id`: The ID of the admin who performed the action.
> - `action`: E.g., 'deactivated_user', 'created_tariff'.
> - `model_type` & `model_id`: Polymorphic references to the modified record.
> - `old_values` & `new_values`: JSON columns storing the state before and after the action.
> - `ip_address`: The IP address of the admin.

6. MULTIPLE CHOICE QUESTIONS (MCQ BANK)

Q1. Which middleware in this Laravel project is responsible for preventing brute-force login attempts?

A) auth

B) guest

C) throttle:5,1

D) verifyCsrf

Correct Answer: C
The `throttle:5,1` middleware limits a user to 5 requests per minute for login and registration routes, preventing automated dictionary/brute-force attacks.

Q2. In which folder are the blade templates of the application stored?

A) app/Views

B) public/views

C) resources/views

D) storage/views

Correct Answer: C
All blade views in a Laravel application are stored in the `resources/views` directory, which Laravel compiles into cached PHP files.

Q3. SDOs trigger the billing process using the `generateBills` method. What happens to the Net Payable amount if a subsidy is applied?

A) Net Payable = Energy + Fixed + Tax + Subsidy

B) Net Payable = Energy + Fixed - Subsidy

C) Net Payable = Max(0, (Energy + Fixed + Tax) - Subsidy)

D) Net Payable = Energy + Tax - Subsidy

Correct Answer: C
The net payable amount adds Energy Charges, Fixed Charges, and Taxes, subtracts the Subsidy Amount, and ensures the total is not negative using `max(0, ...)`.

Q4. How does the lineman submit readings to the database? Which table is populated?

A) users

B) connections

C) meter_readings

D) bills

Correct Answer: C
When a lineman submits a reading, the system populates the `meter_readings` table, recording the current unit value, GPS lat/lng, and physical photo path.

Q5. Which Laravel validation rule guarantees that no two users can register with the same email address?

A) required

B) email

C) unique:users

D) distinct

Correct Answer: C

The `unique:users` validation rule queries the database to verify that the email address is not already present in the `email` column of the `users` table.

6. MCQ BANK & SILLY QUESTIONS

Q6. What happens when the SDO approves a connection in OfficerController.php?

A) A bill is instantly generated

B) A sequential meter number starting with "MT-" is assigned using a DB transaction lock

C) The farmer is immediately logged out

D) The connection status is set to 'pending'

Correct Answer: B
Upon SDO approval, a sequential meter ID like `MT-10001` is assigned to the connection inside a transaction using a database row lock.

Q7. What is the database engine recommended for this Laravel project in the environment settings?

A) SQLite

B) MySQL 8.0

C) PostgreSQL

D) MongoDB

Correct Answer: B
The project configuration is set up for **MySQL 8.0**, which handles e-governance transactions and relational indexing.

7. SILLY & NORMAL QUESTIONS (FAQ)

Q1. What is the `.env` file, and why is it excluded from Git?

> The `.env` (Environment) file stores local system configurations, such as database credentials, SMTP server details, and Razorpay API secret keys. It is excluded from Git to prevent sensitive keys from being exposed in public repositories.

Q2. What happens if I delete the `vendor` folder? How do I restore it?

> The `vendor` folder contains all framework files and third-party PHP packages (e.g., DomPDF, Razorpay SDK). If deleted, the application will crash. You can restore it by running:
>
> ```
> composer install
> ```
>
> This reads the `composer.json` and `composer.lock` files to download the correct package versions.

Q3. Why does the page style look broken when I clone the project on a new system?

> This project uses Tailwind CSS assets compiled using **Vite**. If the assets are not compiled, the CSS will fail to load. To fix this, run:
>
> ```
> npm install
> npm run build
> ```
>
> This compiles the CSS and JS files into the public directory, restoring the layouts.

Q4. What is `php artisan key:generate` used for?

> This command generates a cryptographically secure 32-character string and updates the `APP_KEY` variable in the `.env` file. Laravel uses this key to encrypt user sessions, cookies, and other encrypted fields. Without it, the application will refuse to start.

Q5. Where is the database name defined? How do I change it?

> The database name is defined in the `.env` file under the key `DB_DATABASE` (e.g., `DB_DATABASE=power_distribution`). You can change this value to point to another MySQL database schema.